这一讲：agent 的监督微调

上一讲片尾

下一讲：agent 的监督微调
把录下来的轨迹变成模型权重

第 4 讲：更新 agent 的三个位置

上下文窗口	✓ 已讲过
外部产物	✓ 已讲过
模型权重	这一讲

② 改权重 换来什么

③ 轨迹变成 token 序列

④ 挑选轨迹

⑤ 统一 数据格式

⑥ 跑训练

⑦ 怎么知道 训好了

⑧ 交给 强化学习

⑨ 课堂问答



课件 p.1–2、p.4

② 改权重

一条轨迹：agent 干一次活录下的全部数据

来源：SWE-Gym · 拿到一个 issue

bash　文件编辑器

沙箱里只有这两样工具

system 脚手架提示词 → user issue 原文 → assistant 想法 + bash → observation 测试输出 → assistant 想法 + 编辑 → observation diff → … → assistant 提交

✓ 仓库测试

system
脚手架的提示词

user
issue 原文

assistant
agent 的想法和命令

observation
命令返回的结果：测试输出、diff

三条 assistant 消息，最后一条是提交；成没成功，由仓库自己的测试决定



课件 p.3（中文重绘）

更新 agent 的三个位置

位置	优点	缺点
上下文窗口	保真度高	贵、噪声大、不会判断什么重要
外部产物	可查看、可编辑、可检索、可移植	要归纳、挑选、维护
模型权重 本讲	推理更快、能大范围改变行为 每次调用都生效 · 提示词不用加	更新慢、不透明、只对这一个模型有效

要跑训练
前两行几秒就能更新

像个黑盒
改完没法打开看

挪不走
换不到另一个模型上



课件 p.4（中文重绘）

训进权重，模型学到了什么？

system
脚手架提示词

user
issue 原文

assistant
想法 + bash

observation
测试输出

assistant
想法 + 编辑

observation
diff

...

assistant
提交

能执行的工具调用

写出这个脚手架真能执行的调用

对话变长也能继续

学的轨迹本来就长

会等工具结果

知道工具结果长什么样

~~自己编结果~~

数据里的其他一切

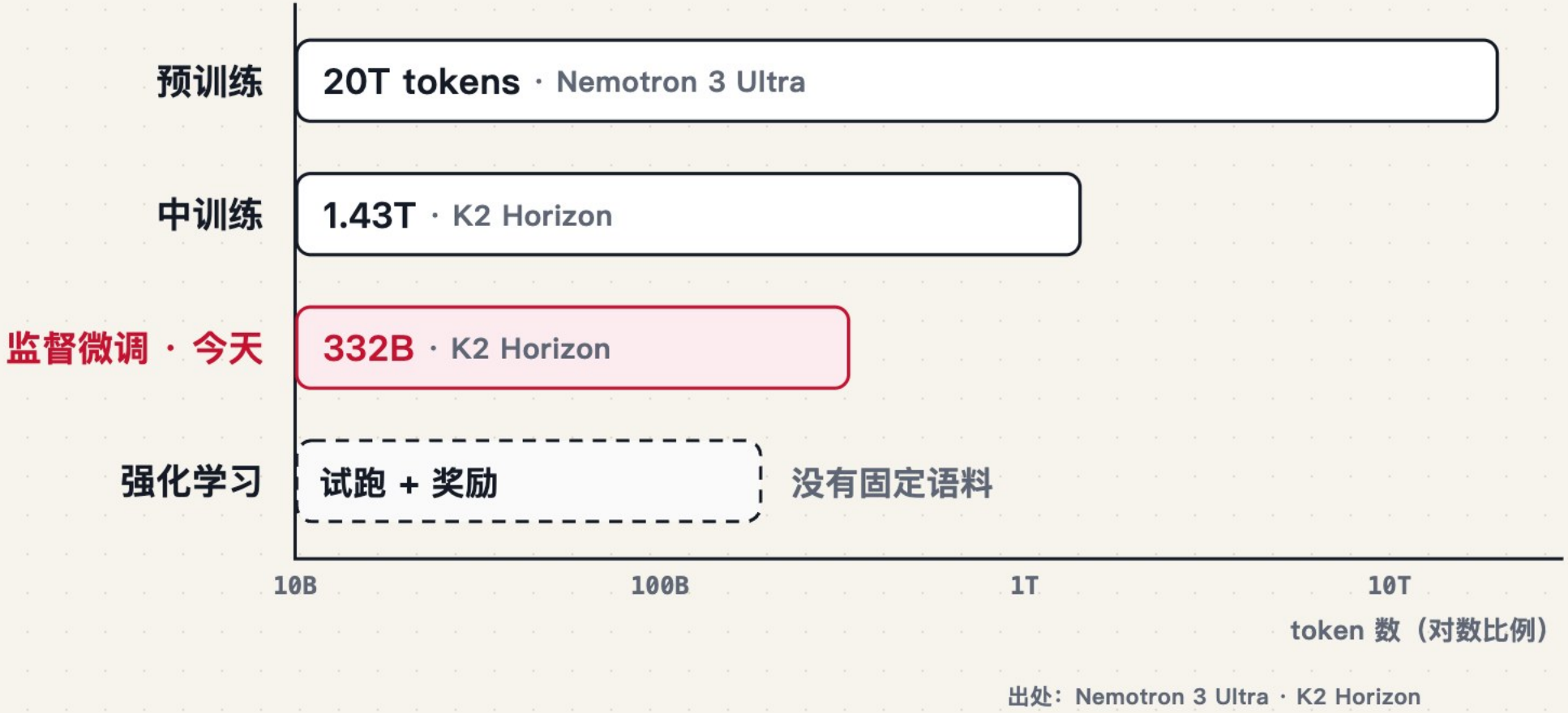
- 多余的注释
- 重复的命令
- 只会一种解法



课件 p.5（中文重绘）

② 改权重

每往下一步，token 更少，筛得更狠



出处：Nemotron 3 Ultra · K2 Horizon



课件 p.6（中文重绘）

② 改权重

Kimi K3 的后训练：先监督微调

监督微调
打下 agent 的
基础能力
调工具 · 做完长任务

→

强化学习
不同推理强度下
练出各领域专家

→

在线策略蒸馏
把专家合成
一个模型

冷启动检查点

> “为后面的强化学习建立一个高质量的冷启动策略”
> —— Kimi K3 §4.1

基座模型 → 直接强化学习 DeepSeek-R1-Zero

首先 we need to 检查这个 function，然后 return 结果… 示意

R1 几千条精选样本 → 监督微调 → 强化学习



课件 p.7（中文重绘）

监督微调和强化学习，放一张表

	监督微调	强化学习
从哪里学	录下来的轨迹，不管谁产生的 可来自更强的模型，甚至模型自己	自己的试跑，由奖励打分
信号	每个 token 都有目标	每次试跑一个数
需要什么	数据和 GPU	活的环境、沙箱、能检验的奖励
擅长	工具调用、格式、长任务	把本来能尝试的事做得更好
什么时候卡住	示范错、太窄、用完了	起点太弱、奖励没法检验



课件 p.8（中文重绘）

整条轨迹 = 一个 token 序列

system 脚手架提示词

user issue 原文

assistant 想法 + bash

observation 测试输出

assistant 想法 + 编辑

observation diff

…

assistant 提交

聊天模板
千问

“·” 是空格，“↵” 是换行 · 蓝框是特殊 token

<|im_start|>tool ↵ 3·failed,·41·passed ↵ <|im_end|> ↵

<|im_start|>assistant ↵ The·test·compares·floats.·Fix·it. ↵

<tool_call> ↵ str_replace(...) ↵ </tool_call> ↵

<|im_end|>

全部表示成一整个字符串，喂进训练框架

system user assistant observation assistant observation … assistant

一个 token 序列 · 每个分界都是一个模板 token

验证器



课件 p.11（中文重绘）

哪些 token 拿来训

system 脚手架提示词	user issue 原文	assistant 想法 + bash	observation 测试输出	assistant 想法 + 编辑	observation diff	…	assistant 提交
0 0 0 0	0 0 0 0	1 1 1 1	0 0 0 0	1 1 1 1	0 0 0 0		1 1 1 1

掩码：每个 token 一位（1 = 这个位置的预测算数，格子数示意）

■ 算损失　□ 只当上下文

交叉熵，只算在助手 token 上

$$L(\theta) = -\sum_{t\,:\,m_t=1} \log p_\theta(x_t \mid x_{<t})$$

降到最小 = 在前面一切的条件下，提高每个录下来的动作的概率

普通语言模型：整段都训



agent 监督微调：只训助手





课件 p.12（中文重绘）

为什么不训观察?

Q 为什么不也训模型去预测观察?

一般经验：只训助手消息

观察、用户内容是环境或用户给的，
不是助手预测出来的。

system · user · observation 灰，只当条件；assistant 红，才是目标

Meta · CWM（课上口头补充）

训模型输出
代码执行的结果

- ✓ **懂执行语义**
  对理解代码怎么跑很有帮助
- ! **占模型容量**
  模型太小，可能伤到别的能力
- ? **偏实验**



课件 p.12（中文重绘）

掩码是怎么造出来的

① 渲染
写成字符串 · 记账

② 分词
字符位置 → token 位置

③ 掩码
助手段 1 · 其余 0

<|im_start|>	user	↵	Fix	·the	·float	·test	.	<|im_end|>	↵	<|im_start|>	assistant	↵	Run	·py	test	.	<|im_end|>
0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1

↵	<|im_start|>	tool	↵	3	·failed	<|im_end|>	↵	<|im_start|>	assistant	↵	Use	·is	close	.	<|im_end|>
0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1

训练器只看到： 一个字符串 + 一串位 ，别的什么都没有 （对话和切法是示意）

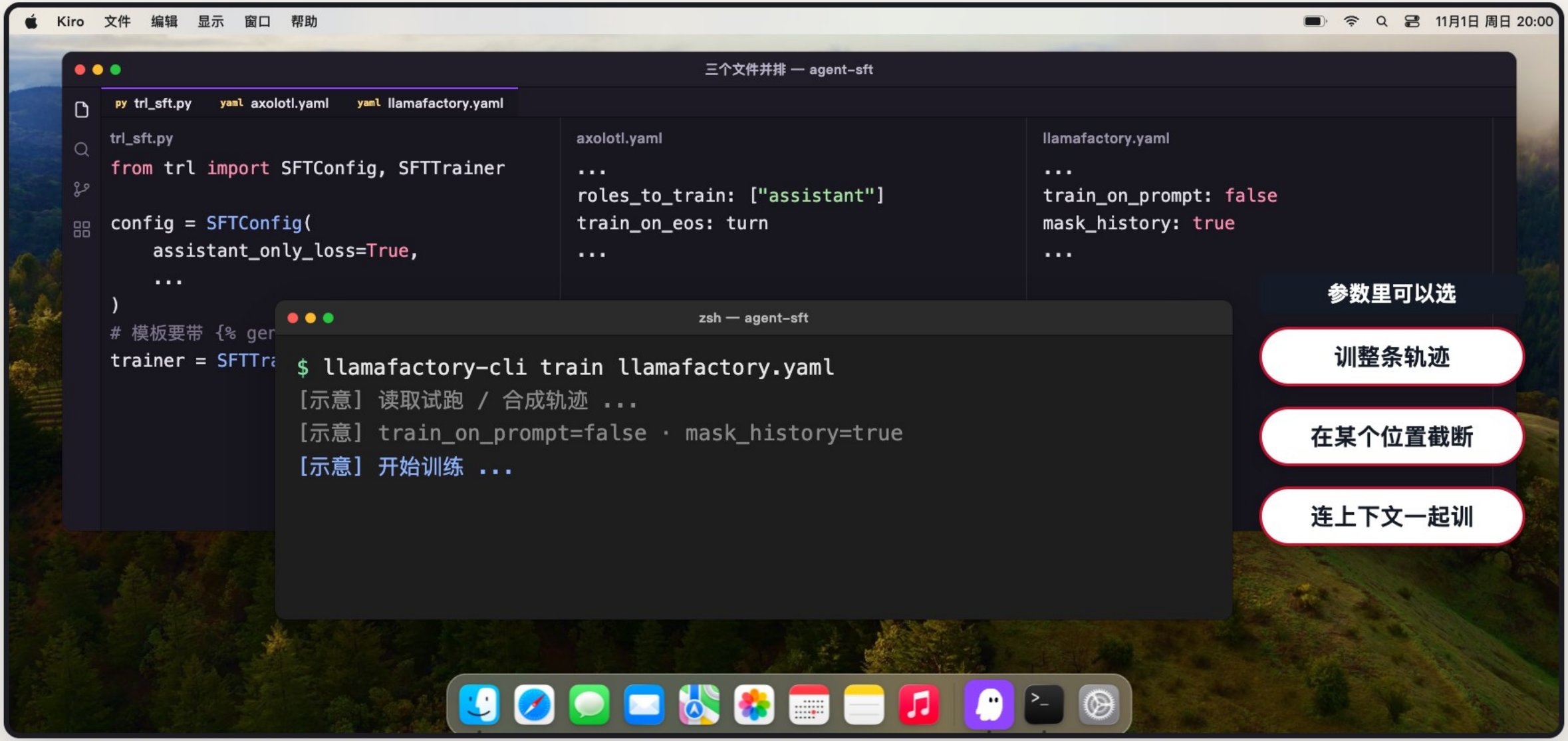


课件 p.13（中文重绘）


Kiro 文件 编辑 显示 窗口 帮助
11月1日 周日 20:00
三个文件并排 — agent-sft
py trl_sft.py
yaml axolotl.yaml
yaml llamafactory.yaml
trl_sft.py
from trl import SFTConfig, SFTTrainer
config = SFTConfig(
assistant_only_loss=True,
...
)
模板要带 {% generation %} 标记
trainer = SFTTrainer(args=config, ...)
只有 TRL 放进了模板：模板要带生成标记
axolotl.yaml
...
roles_to_train: ["assistant"]
train_on_eos: turn
...
llamafactory.yaml
...
train_on_prompt: false
mask_history: true
...
打开 mask_history：只训最后一轮
同一个决定，只有 TRL 放进了模板



课件 p.49（备用页）


Kiro 文件 编辑 显示 窗口 帮助
11月1日 周日 20:00
三个文件并排 — agent-sft
py trl_sft.py
yaml axolotl.yaml
yaml llamafactory.yaml
trl_sft.py
from trl import SFTConfig, SFTTrainer
config = SFTConfig(
assistant_only_loss=True,
...
)
模板要带 {% ge
trainer = SFTTra
axolotl.yaml
...
roles_to_train: ["assistant"]
train_on_eos: turn
...
llamafactory.yaml
...
train_on_prompt: false
mask_history: true
...
zsh — agent-sft
$ llamafactory-cli train llamafactory.yaml
[示意] 读取试跑 / 合成轨迹 ...
[示意] train_on_prompt=false · mask_history=true
[示意] 开始训练 ...
参数里可以选
训整条轨迹
在某个位置截断
连上下文一起训



课件 p.49（备用页）

结束符：必须在掩码里

系统给的 · 不是训练目标

<|im_start|>assistant ↵ I'll·run·the·failing·test. ↵ <tool_call> ↵ bash(cmd="pytest·-q") ↵

</tool_call> ↵ <|im_end|> ↵ ← 模型自己产生，必须在掩码里

“·”是空格，“↵”是换行 算损失 只当上下文

一轮结束 = 结束这条消息 / 交给工具 / 做完整个任务

Kimi K3 的模板：用不同的 token 把它们分开

[open]message·role=assistant[sep] [open]think[sep] … [close]think[sep] [open]response[sep] Use·math.isclose.

[close]response[sep] [end_of_msg]



课件 p.15（中文重绘）

结束符：必须在掩码里

系统给的 · 不是训练目标

<|im_start|>assistant ↵ I'll·run·the·failing·test. ↵ <tool_call> ↵ bash(cmd="pytest·-q") ↵

</tool_call> ↵ <|im_end|> ↵ ← 模型自己产生，必须在掩码里

“·”是空格，“↵”是换行 算损失 只当上下文

千问

<|im_end|>

课件 p.15

Kimi K3

[end_of_msg]

课件 p.15

Llama

<|eot_id|>

示意

⚠ 实践中常见：把 算损失 和 只当上下文 弄混



课件 p.15（中文重绘）

推理块：两个开关分开设

<|im_start|>assistant ↵ <think> ↵ The·assert·compares·floats. ↵ </think> ↵ Use·math.isclose. ↵ <|im_end|>

课件 p.16 的设法：推理只当上下文，答案和结束符算损失 ■ 算损失 □ 只当上下文

推理 （长度示意）

The·assert·compares·floats.·So·…·then·…·maybe·…·so·…·…

工具调用

Use·math.isclose.

全训：模型容易只学会写长推理，和结果混在一起



课件 p.16（中文重绘）

BalanceSFT：让推理和执行平衡

SSB

学出来的权重：在损失里重新平衡
推理 token 和工具调用 token

HDR

对难样本重新采样

BalanceSFT 表 5 · BFCL · 照原图重绘

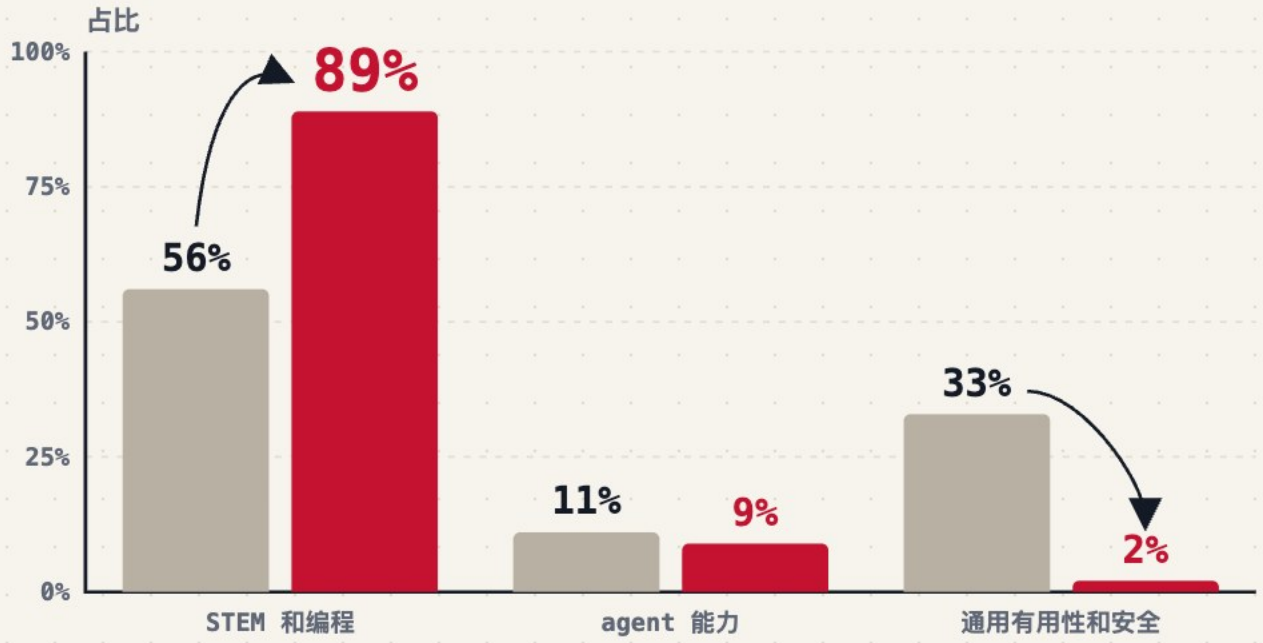
基座模型	SSB	HDR	单轮	多轮
Qwen2.5-Coder-Inst+SFT	×	×	81.73	38.25
	✓	×	82.20	41.62
	×	✓	83.85	43.12
	✓	✓	84.00	47.00
Llama-3.2-3B-Inst+SFT	×	×	73.67	33.25
	✓	×	76.82	34.62
	×	✓	78.28	35.75
	✓	✓	78.99	36.12

长推理主导了损失，要做对的是短短的工具调用



课件 p.17（中文重绘）

按条数配，按 token 算



MAI-Thinking-1 表 10 · 合并阶段的监督微调

按条数配完，回头看 token

- 样本占比：他们觉得每种能力多重要
- token 占比：损失真正花在哪



课件 p.18（中文重绘）

监督一条轨迹：还没解决的问题

? 没有公开实验：在 agent 监督微调里改掩码，再测量会发生什么

? 做得仔细的掩码实验，都是单轮指令微调，不是 agent

? 不遮提示有帮助的唯一情况

× 提示长　× 回答短　× 数据少　agent 轨迹：三项都相反

? 8B 规模上这个实验很便宜，却一直没人做



课件 p.19（中文重绘）

轨迹从哪里来：谁跑的 × 留哪些

谁跑的任务	全部留下	只留通过的
更强的模型跑	普通蒸馏	SWE-Gym · Nemotron · OT-Agent 老师比学生强 · 测试筛选，很便宜
模型自己跑	MAI · 自己的推理试跑	专家迭代 只留通过的，再重复 · 后面几讲

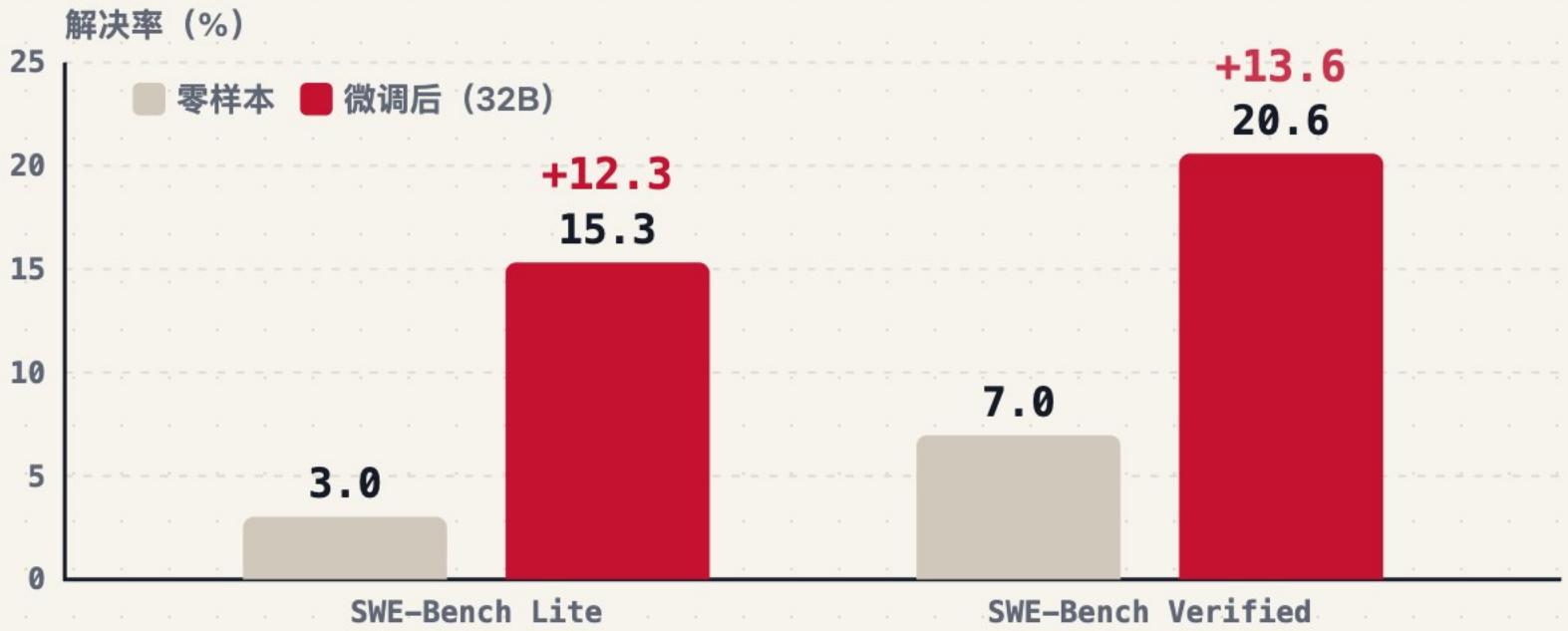
大多数 agent 监督微调的数据，在右上格



课件 p.21（中文重绘）

从更强的模型蒸馏：491 条轨迹，涨十多个点

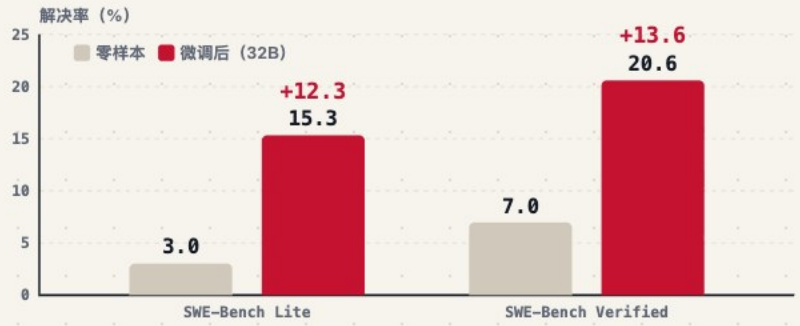
GPT-4o · Claude → SWE-Gym 任务 → ✓ 仓库测试 → **491 条通过** → Qwen 32B





课件 p.22（中文重绘）

从更强的模型蒸馏：491 条轨迹，涨十多个点

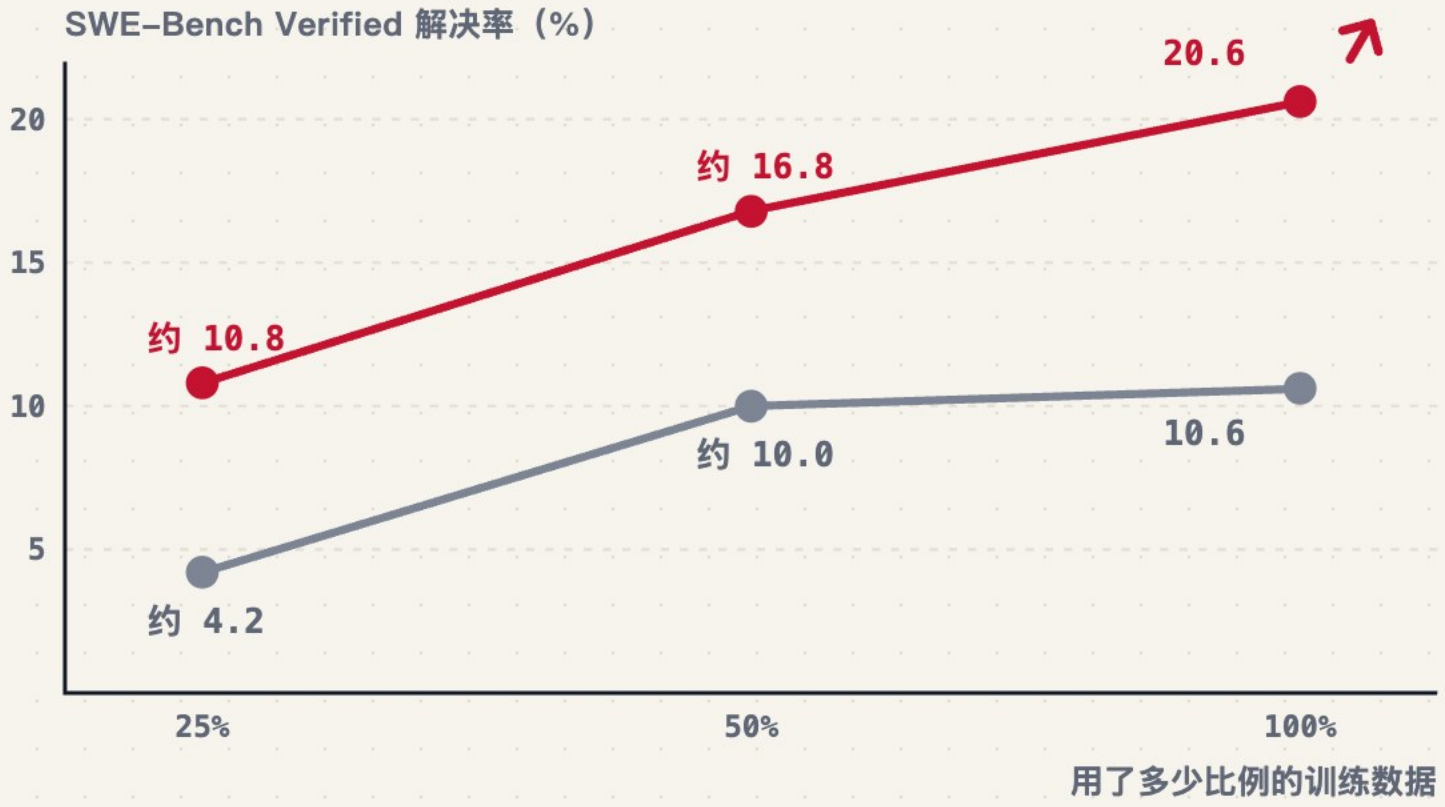


模型	空补丁（%，↓）			卡在循环里（%，↓）			平均轮数			解决率（%，↑）		
	零样本	微调后	Δ	零样本	微调后	Δ	零样本	微调后	Δ	零样本	微调后	Δ
SWE-Bench Lite（300 题）												
7B	40.3	29.7	−10.7	47.0	31.0	−16.0	20.3	22.2	+1.9	1.0 ±1.0	10.0 ±2.4	+9.0
14B	49.7	18.1	−31.6	31.7	27.1	−4.6	23.2	21.4	−1.8	2.7 ±1.9	12.7 ±2.3	+10.0
32B	27.0	18.1	−8.9	16.7	18.1	+1.5	15.5	29.3	+13.9	3.0 ±1.4	15.3 ±2.5	+12.3
SWE-Bench Verified（500 题）												
7B	45.8	33.8	−12.0	39.6	21.0	−18.6	21.9	35.3	+13.4	1.8 ±1.1	10.6 ±2.1	+8.8
14B	44.9	14.5	−30.4	32.1	21.3	−10.7	25.5	30.1	+4.6	4.0 ±1.6	16.4 ±2.0	+12.4
32B	9.5	13.8	+4.3	29.4	23.8	−5.6	24.6	31.6	+7.0	7.0 ±1.3	20.6 ±2.1	+13.6



课件 p.22（中文重绘）

同一批任务，只改轨迹条数



32B

7B

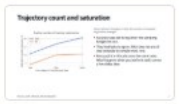
预算用完时还在涨

任务还有富余，用完的是采样算力

7B 一半之后几乎平：小模型更早饱和

点位按原图读数
100% 两点和表 3 一致

每个点都是同一批任务 · 加新任务会怎样 → 本段后面再讲



课件 p.23（中文重绘）

只留通过的，也会带上坏习惯

读了，从没改

改—测循环

补丁里留着 print(

system
脚手架提示词

user
issue 原文

assistant
想法 + bash

observation
测试输出

assistant
想法 + 编辑

observation
diff

…

assistant
提交

算损失 不算

✓ 测试全部通过

损失照抄
每个动作

学生模型学到的

读了不改 改—测循环 补丁里留 print(+ 修复本身

“直接拿来做监督微调数据，会教给模型不想要的行为”

—— Nemotron 3 Ultra §3.1.1 · 成功只说明最终做成了，中间过程没法衡量



课件 p.24（中文重绘）

按形状过滤：数轮数就够好

排名	过滤方式	SWE-bench Verified (100)	OT-TBLite	Terminal-Bench 2.0	平均 · 原始	归一化
1	至少 5 轮	29.00	19.10	11.61	19.90	+1.25
2	过滤掉超时的	26.67	18.03	10.49	18.39	-0.35
3	过滤掉子 agent 的轨迹	23.00	17.31	10.86	17.06	-0.90

短轨迹可能是……

✓

一次就做对了

学不到出错后怎么恢复

✗

轨迹本身是错的

也可能是一条错误轨迹

⚠

生成时出了错

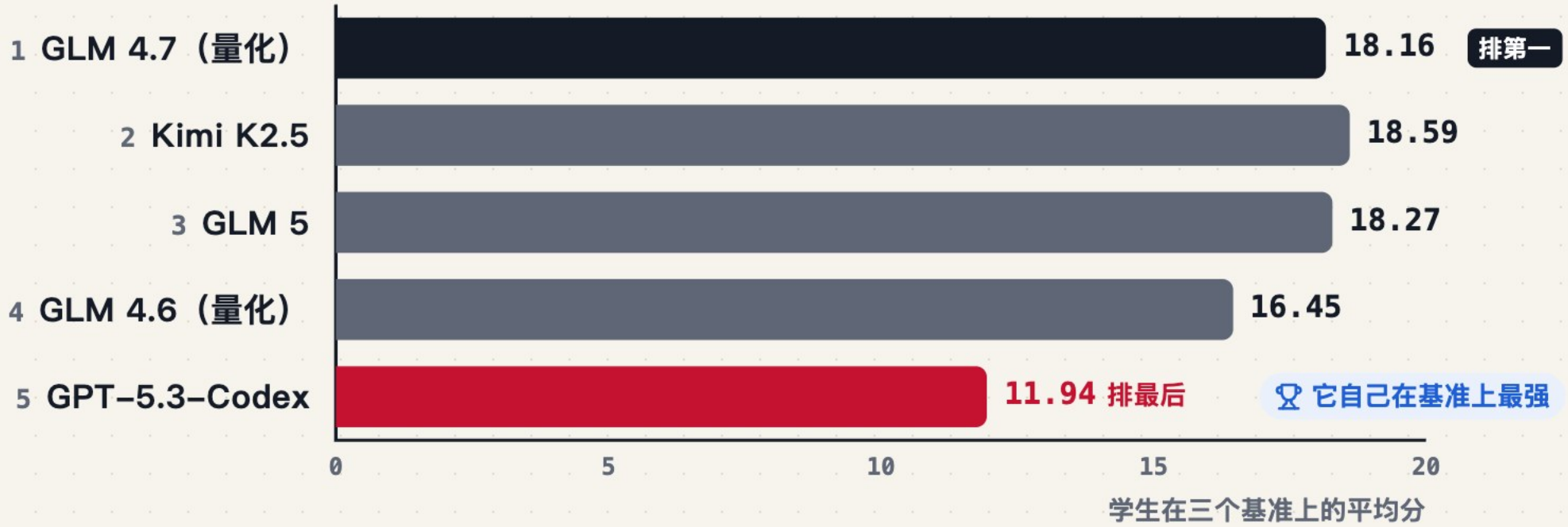
所以才那么短



课件 p.25（中文重绘）

更强的模型 ≠ 更好的老师

按论文的归一化排名从上往下排 · 条长 = 学生平均分（原始）



老师 A → 学生 A ✓ 留下　　老师 B → 学生 B ×

比看排行榜靠谱，成本也低



课件 p.26（中文重绘）

任务来源：整条流程里差距最大的一环

排名	任务来源（95 个里节选）	SWE-bench Verified (100)	OT-TBLite	Terminal-Bench 2.0	平均 · 原始
1	SWESmith	32.33	17.63	~~6.37~~	18.78
2	StackExchange SuperUser	13.33	16.68	10.86	13.62
3	StackExchange Tezos	16.33	16.94	9.36	14.21
4	IssueTasks	24.00	16.44	6.74	15.73
		…			
95	AgentTuning-OS	0.00	5.64	0.37	2.00

排前面的：修 issue 的任务 · 人写的基础设施问题；不同来源帮的是不同的基准

95 个任务来源 → 先在目标基准上试 → 混最好的 4–8 个

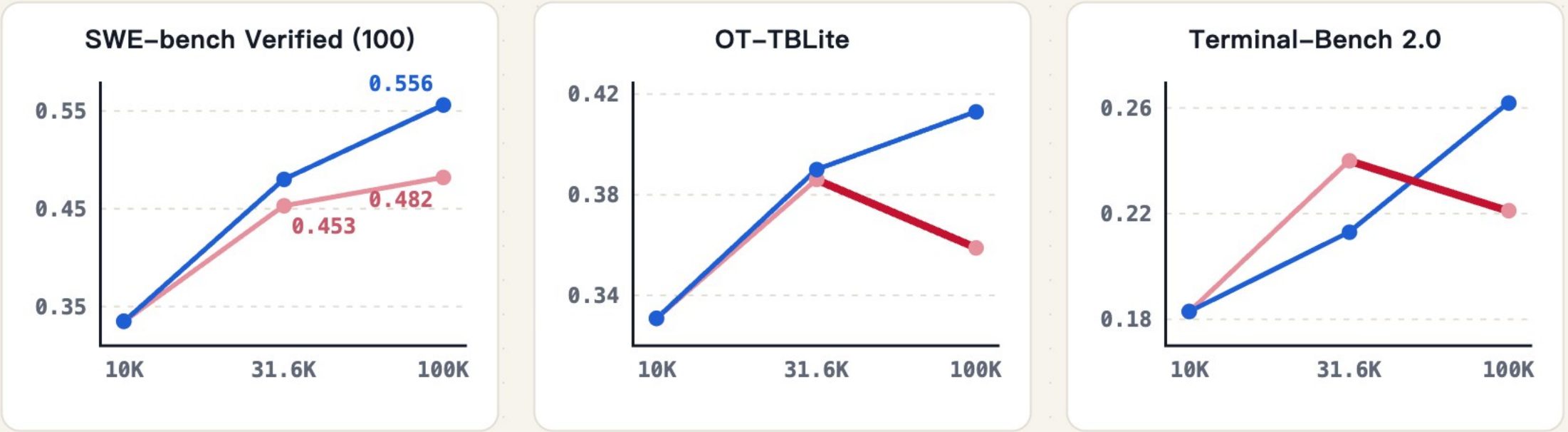
再往宽了混：没帮助



课件 p.27（中文重绘）

加新任务，才一直往上爬

点位按原图读数



横轴：数据集大小（行，从同一个 1 万条出发） · 纵轴：准确率　多采试跑（同样任务）　合成扩充（加新任务）

同一个分布里多采 → 饱和，甚至下降

扩大分布 → 继续往上走



课件 p.28（中文重绘）

课堂讨论：同一个任务跑回来三条轨迹，留哪些？

C 11 次没改变，之后才解决

训练集

A 短：4 轮就解决了 issue

全程都对 · 没见过恢复

示意

B 长：改错 → 测试失败 → 恢复

✓ 会恢复

⚠ 多余操作 · 学到错的编辑

A 和 B 都要：
让数据覆盖对目标基准最有用的能力



课件 p.29（中文重绘）

课堂讨论：同一个任务跑回来三条轨迹，留哪些？

C 11 次没改变，之后才解决

至少 5 轮

训练集

A 短：4 轮就解决了 issue

被扔掉 · 判错了

示意

B 长：改错 → 测试失败 → 恢复

会恢复

多余操作 · 学到错的编辑

A 和 B 都要：
让数据覆盖对目标基准最有用的能力



课件 p.29（中文重绘）

⑤ 一种格式装下所有数据集

数据集	类别	条数	来源	说明
AgentInstruct	C T W	1.9K	合成	浏览、数据库、操作系统等混合
Code-Feedback	C	66.4K	人工	带运行时反馈循环的代码生成
CodeActInstruct	C	7.1K	合成	带执行的代码生成和工具调用
Go-Browse	W	9.5K	试跑	结构化探索的网页试跑
Mind2Web	W	1.0K	人工	真实网站上的人工网页示范
Nebius SWE Trajectories	S	13.4K	试跑	只用开放权重模型的 SWE-agent 轨迹
NNetNav-live	W	5.0K	试跑	事后标注的真实网页探索
NNetNav-wa	W	4.2K	试跑	事后标注的 WebArena 探索
openhands-feedback	C T W	0.2K	试跑	带人工反馈的 OpenHands agent 轨迹
Orca AgentInstruct	T	1046.1K	合成	大规模合成的工具调用指令
SWE-Gym	S	0.5K	试跑	解决真实 GitHub 仓库任务的轨迹
SWE-smith	S	5.0K	合成	合成修 bug 任务上的轨迹
Synatra	W	99.9K	合成	合成的网页教程示范

Agent Data Protocol 表 1 · 共 13 个数据集



课件 p.31（中文重绘）

同一个网页，两种存法

同一个网页：google.com 示意

Go-Browse

W · 9.5K · 试跑

存无障碍树

无障碍树

```
[1] RootWebArea 'Google'
  [2] link 'Gmail'
  [3] combobox '搜索'
```

→ 定制转换器 →

Mind2Web

W · 1.0K · 人工

存 HTML

HTML

```
<html>
  <title>Google</title> …
</html>
```

→ 定制转换器 →

OpenHands

要训的 agent

每两个之间都要写一个转换器：工程量很大

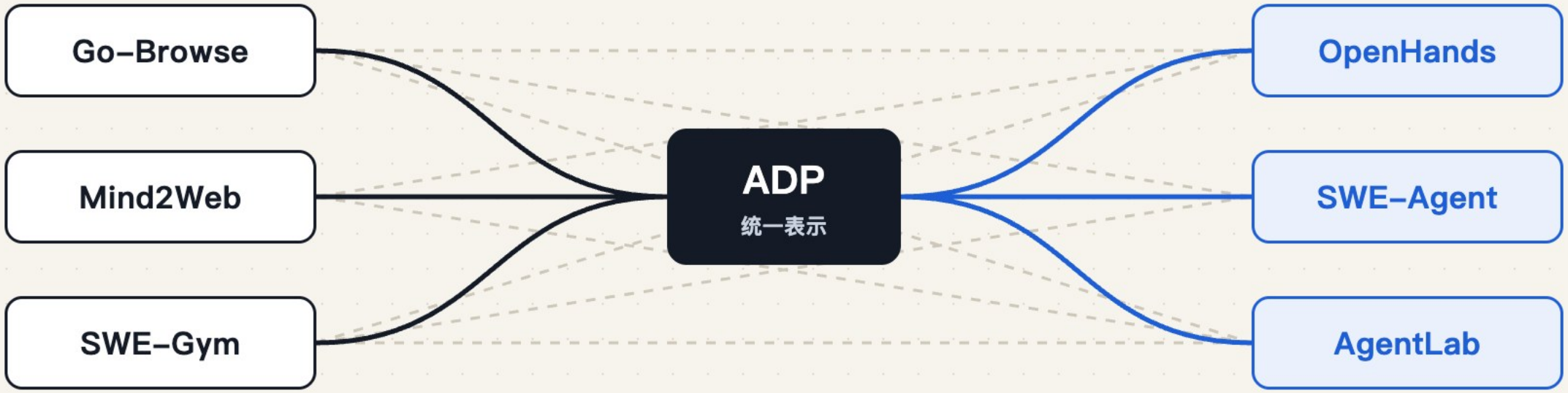


课件 p.31（示意）

每边转一次，不用每一对都转

数据集：每个转一次

脚手架：每个转一次



× 不用每一对写一个转换器



课件 p.32（中文重绘）

同一条轨迹，按脚手架写出来

同一条 ADP 轨迹

OpenHands 示意

系统提示词

想法：先打开搜索页

```
BrowseURLAction(url="google.com")
```

上下文太长怎么办

✓ 工具调用能解析

SWE-Agent 示意

系统提示词

想法：先打开搜索页

```
open_url google.com
```

上下文太长怎么办

✓ 每个调用带一段想法

AgentLab 示意

系统提示词

想法：先打开搜索页

```
goto("google.com")
```

上下文太长怎么办

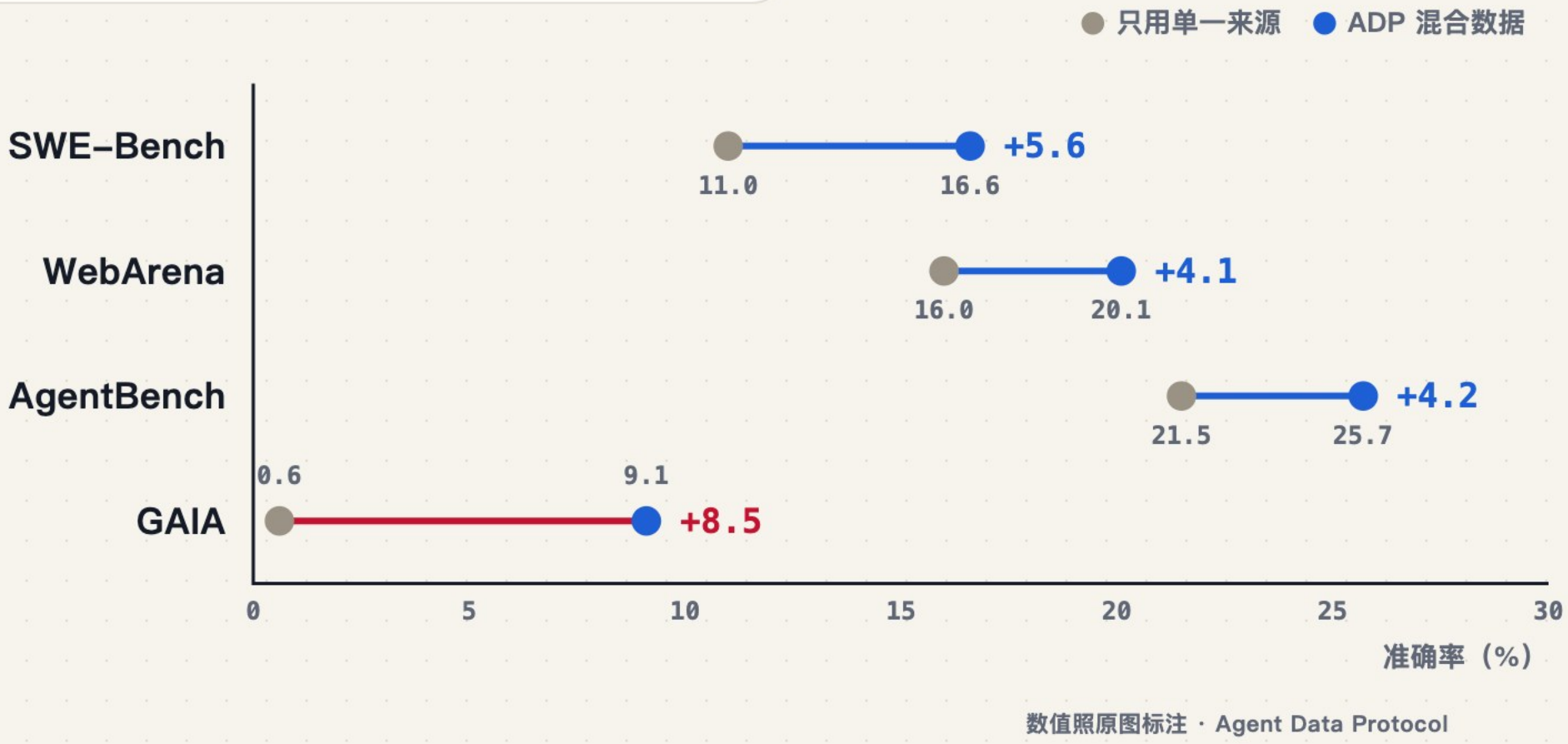
✓ 对话按该有的方式结束



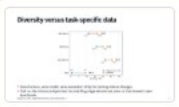
课件 p.33（中文重绘）

混合数据赢过单一领域，哪怕在那个领域自己的基准上

同样的脚手架 · 同样的模型 · 同样的评测，只改训练数据配比



数值照原图标注 · Agent Data Protocol



课件 p.34（中文重绘）

Kiro 文件 编辑 显示 窗口 帮助 11月1日 周日 20:00

sft_configs.yaml — agent-sft 示意

yaml sft_configs.yaml

```yaml
# 示意 · 照课件 p.36 的三行表改写
nemotron_3_ultra:
  seq_len: 294912 → 515000  # packed
  batch: 64
  lr: 1.5e-5 → 1e-6
mai_self_distill:
  seq_len: 128k  # packed
  batch: 2048
  lr: 1.7e-5 → 5.2e-6
  warmup: 2%
k2_horizon:
  seq_len: 512K
  stages: 3
  lr: decayed on a high-quality subset
```

zsh — agent-sft

```
$ train --config sft_configs.yaml
[示意] 学习率随训练变化（只画趋势）
nemotron    1.5e-5 ▬▬▬▬▬▬▬▬▬▬ 1e-6
mai         1.7e-5 ▬▬▬▬▬▬▬▬▬▬ 5.2e-6
k2                 ▬▬▬▬▬▬▬▬▬▬ 子集上衰减
[示意] 三家的学习率都随训练衰减
```

课上有人问

这是全量训练吗？ 是

和预训练一样，只是数据不同

监督微调 ▢▢■■▢■ 遮掉非 agent 部分

预训练 ■■■■■■ 不遮



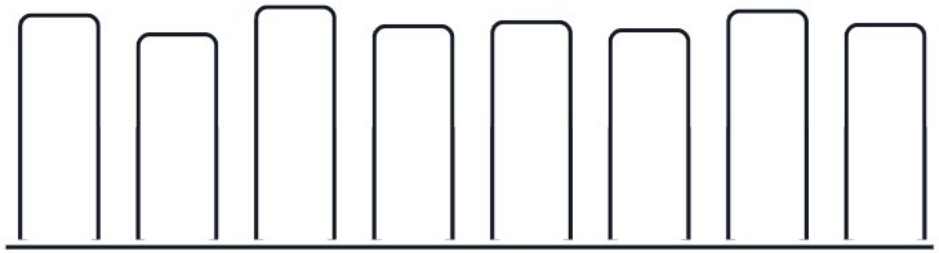
课件 p.36（中文重绘）

混合专家模型：少数专家吃掉大部分 token

前沿模型：混合专家　公开论文：稠密模型

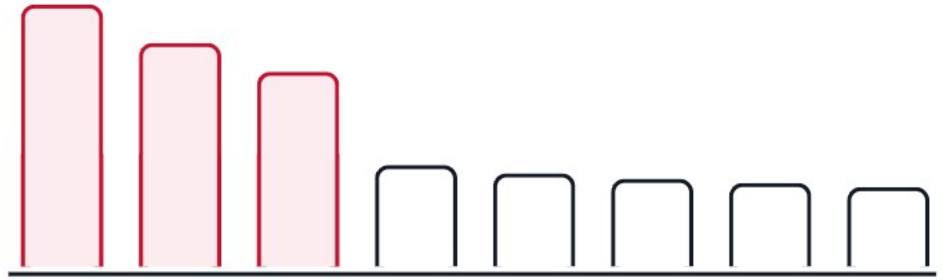
示意，柱子没有数

宽泛的预训练数据



每个专家分到的 token

窄的 agent 监督微调数据



每个专家分到的 token

开训前读一两篇技术报告

前沿实验室的经验很多能直接用上

路由均衡 ×1000

对比强化学习阶段

`dropout 0.15`



课件 p.37（中文重绘）

打包：一条定长序列里塞几段完整对话

块的宽度按原图比例 · 按 Nemotron 3 Ultra 报告改画



→ 放进剩余容量最贴合的那一包

不切开、不截断：为防止幻觉

相同提示不放一包 · 打完包再打乱



课件 p.38（中文重绘）

模板漂移：在工具循环里是致命的

训练用的模板 = 部署用的模板 · 连空白都一字不差

渲染 ① 后面不接工具消息

token 内容示意

`<|im_start|>assistant` ↵ `<think>想法</think>` ↵ `<tool_call>…</tool_call>` `<|im_end|>`

相同前缀

渲染 ② 后面接一条工具消息

`<|im_start|>assistant` ↵ `<tool_call>…</tool_call>` `<|im_end|>` `<|im_start|>tool` 测试输出 `<|im_end|>`

思考块变了，前缀对不上

> **“聊天模板应该始终和模型训练时的格式一致。”**
>
> —— Transformers 文档 · writing chat templates



课件 p.39（中文重绘）

想学跑训练，读公开的训练产物

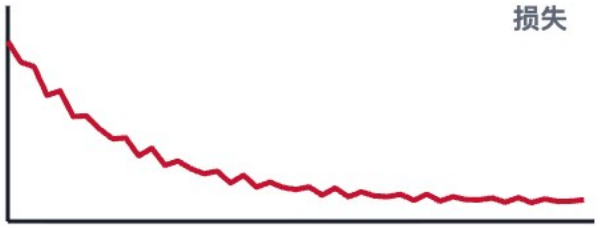
K2 Horizon 模型卡

分阶段的训练表 示意

阶段	步数	token 数	序列长度	用途
1	…	…	…	…
2	…	…	…	…
3	…	…	…	…

训练记录

Weights & Biases 上可看 示意



中间检查点

每个阶段都发布

阶段 1 → 阶段 2 → 阶段 3

可以接着训

Nemotron 3 Ultra

一起发布了

后训练检查点　训练数据　训练配方

至少读一份



课件 p.40（中文重绘）

训练和运行之间的错位

示范里的

同一个前缀
issue + 第一次编辑

编辑 A
录下来的那条
打分

测试通过
预料中的观察
打分

提交
任务完成
打分

它自己跑

编辑 B
合理，但不一样

测试失败
没预料到的观察

现在怎么办?
数据里没有

训练从没到过的状态

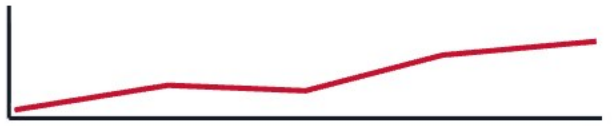
损失低 ≠ 能恢复

DAgger
Ross et al., 2011 · 早就讲过这个问题

损失曲线 · 监控训练正不正常 示意



评测分数 · 真正要看的 示意





课件 p.42（中文重绘）

Kiro 文件 编辑 显示 窗口 帮助 11月1日 周日 20:00

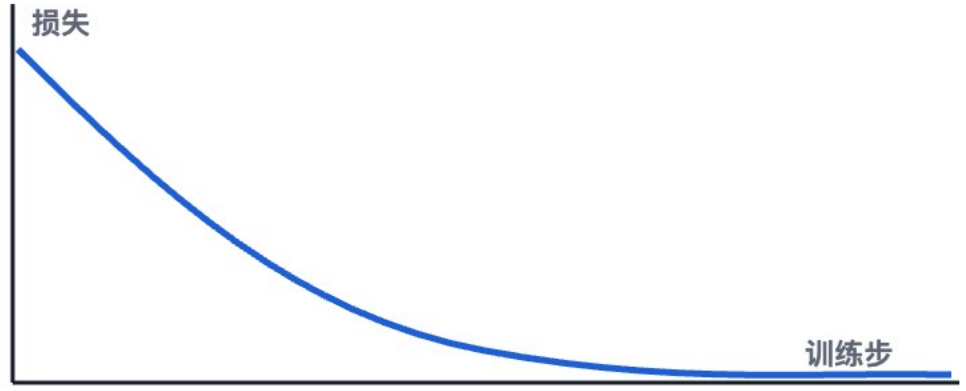
overfit_20.yaml — agent-sft 示意

yaml overfit_20.yaml

```yaml
# 示意 · 先过拟合 20 个样本
model_name_or_path: ...
template: ...
dataset: ...
max_samples: 20
num_train_epochs: ...  # 反复训
...
```

✓ 过关 → 放进脚手架评测

损失：应该能压到接近零 示意



zsh — agent-sft

```
$ llamafactory-cli train overfit_20.yaml
[示意] 同一批 20 条轨迹，反复训 ...
[示意] loss ↓ ↓ ↓ 压到很低
loss 不降 → 查模板、掩码、数据
[示意] 在脚手架里评测：结果能检验的任务
```

降不下来 → bug 在上游，后面的分数都不可信

模板 掩码 数据



课件 p.43（中文重绘）

读别人的数字，做自己的划分

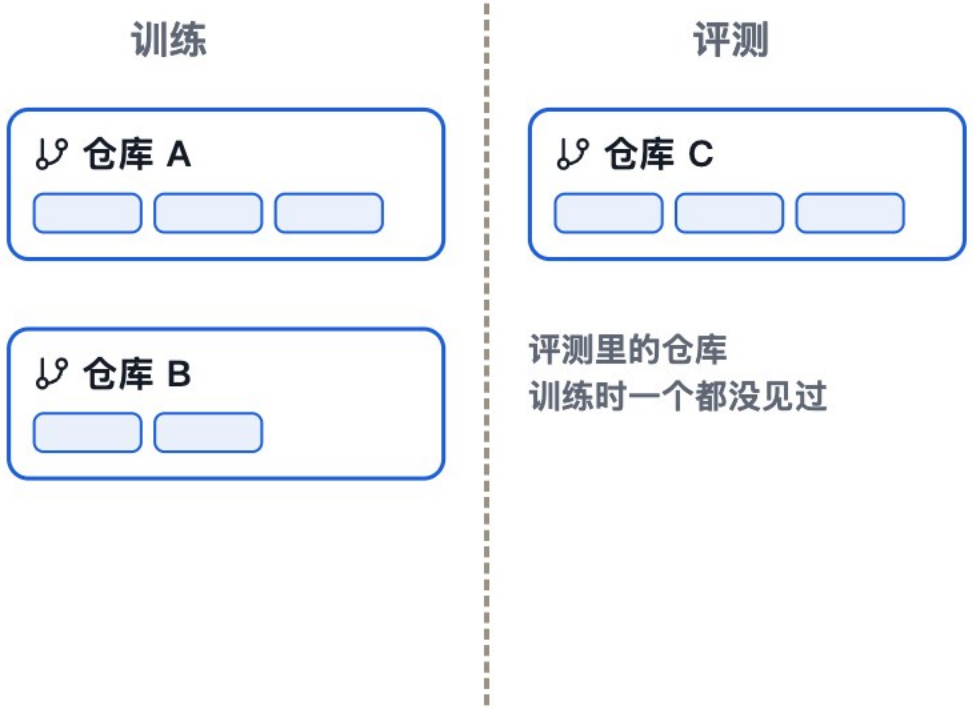
公开的数字 分数条示意



→ 公开结果：有时是挑出来最好的那个

而不是：一个脚手架上跑一次

划分要和结论对得上

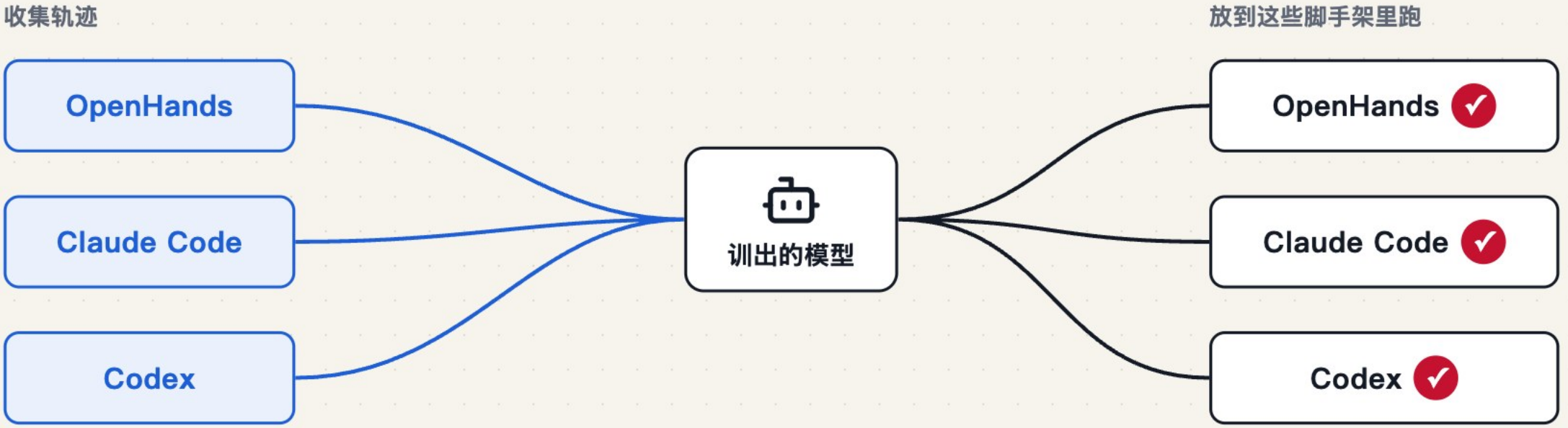




课件 p.43（中文重绘）

换个脚手架，还行不行

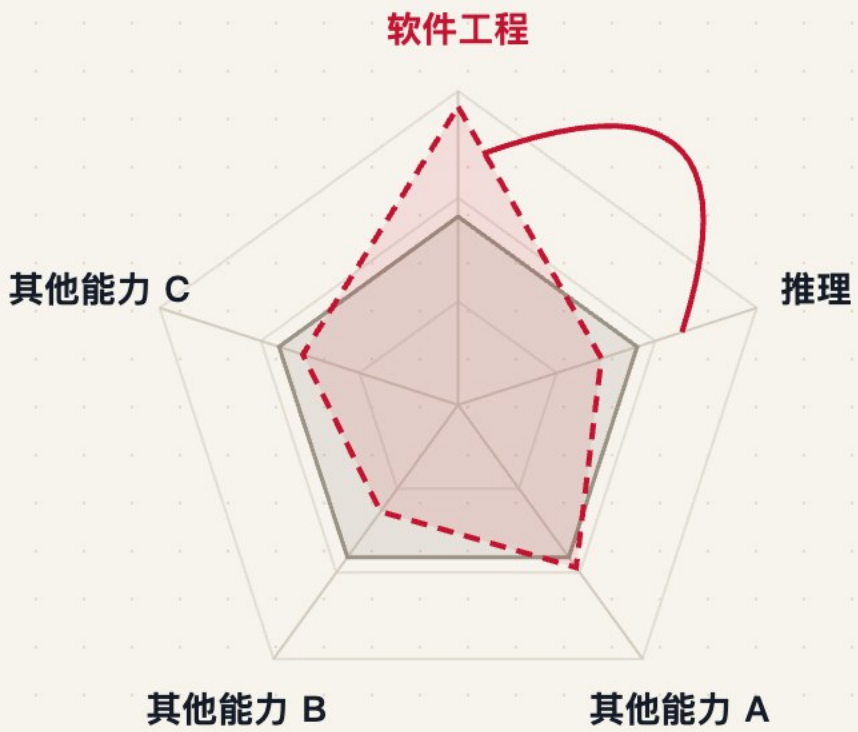
在一个系统里收集的轨迹，会把这个系统的习惯一起教给模型





课件 p.44（中文重绘）

目标领域外的退化



雷达图是示意，不标数

训一种能力，别的也会跟着变

不管你测没测

能调的是数据配比

按样本数平衡 ≠ 按 token 数平衡（见第 ③ 部分）

没打算提升的能力，也要评测

别只在自己关心的那一个领域刷榜

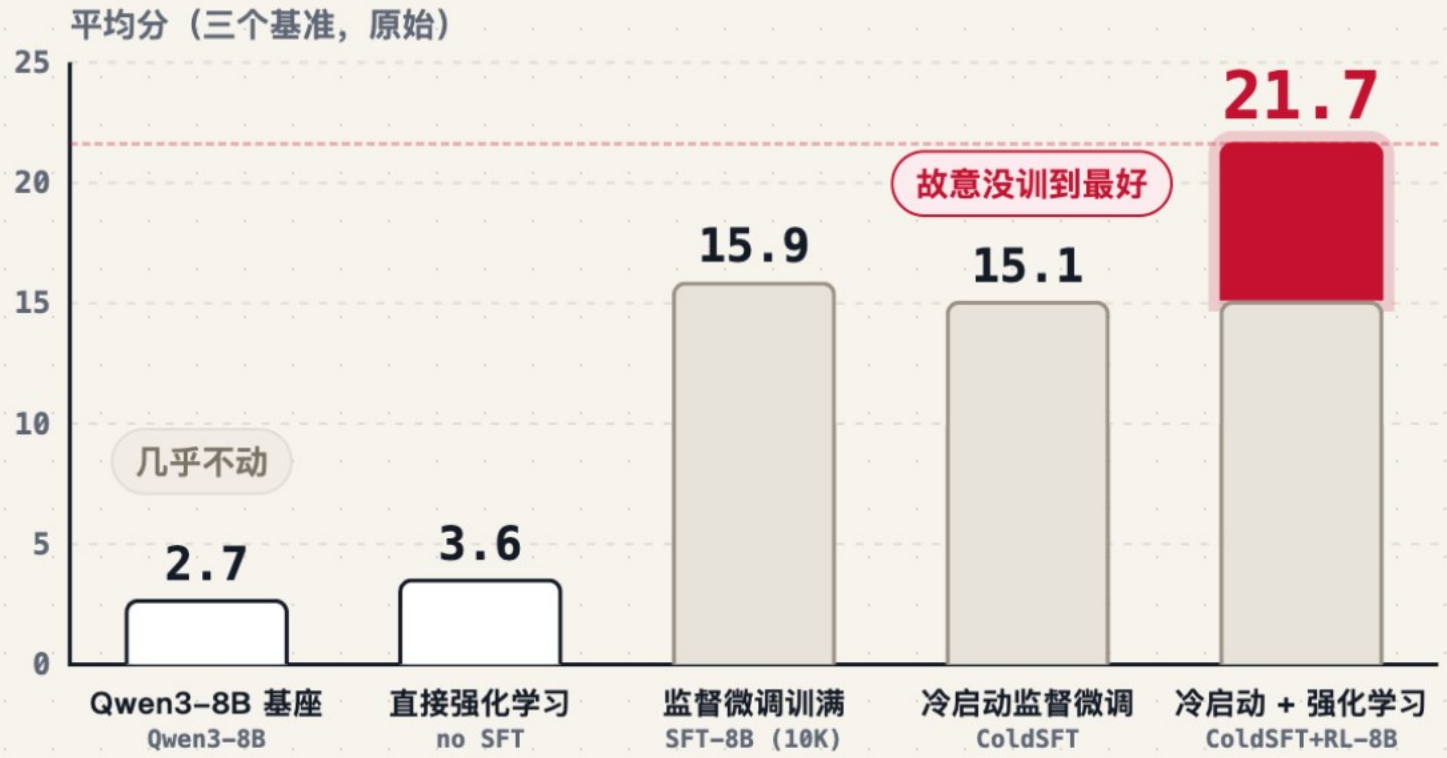
要软件工程强，大概也该训推理

这些能力会迁移



课件 p.45（中文重绘）

强化学习之前，监督微调做多少



OpenThoughts-Agent 表 11 · 三个基准：SWE-bench Verified (100) · OT-TBLite · Terminal-Bench 2.0

> “监督微调模型是带着强化学习的目的去挑的时候，强化学习带来的收益最大。”
>
> —— MAI-Thinking-1 §3.1.4

监督微调 = 一个先验

监督微调（最通用的能力） → 强化学习（具体领域 · 更好的推理）

DeepSeek-R1-Zero 的教训

不做监督微调就上强化学习，推不动基座模型（见第 ② 部分）



课件 p.47（中文重绘）

Q1 没有更强的模型，轨迹从哪来？

课上口头补充

办法一

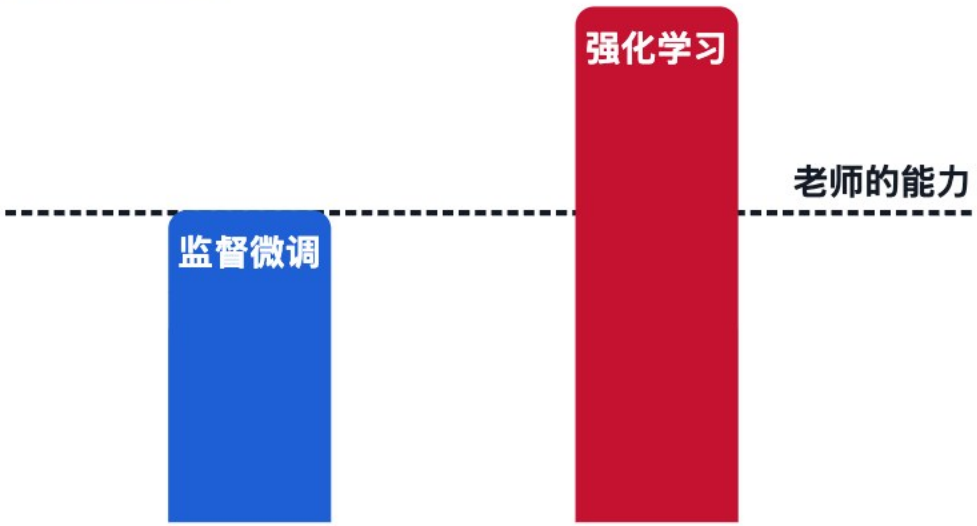
从自己蒸馏

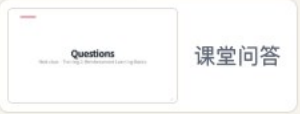
MAI：用它自己的推理去试跑

办法二

更多靠强化学习

领跑的实验室获得 agent 能力的主要办法





课堂问答

Q2 谁都还没有这种模型时，最开始怎么训?

课上口头补充

一开始

大量数据、海量轨迹

再加上不依赖现成轨迹的
强化学习

专有模型具体怎么做：不确定

开源模型的例子

K2 Horizon

监督微调 → 强化学习 → 好几轮微调



课堂问答

Q3 现在还用 LoRA 这种低秩适配吗?

资源充足的实验室

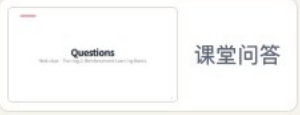
更喜欢全量微调

所有权重一起更新

要省资源时

为了效率用过 LoRA

只训一小部分低秩参数



课堂问答

Q4 错误的步骤要不要监督?

一条犯错又恢复的轨迹（示意）

中间步骤 | 错的那次编辑 | 恢复 | 对的结果

底线

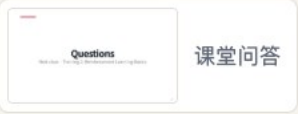
最终结果要对

不希望编程 agent 最后写出错的代码

错的那次编辑也监督

从错误里学东西

对的结果和恢复的过程都要监督



课堂问答

Q5 错的轨迹和对的轨迹，有没有一个比例?

一般靠实验定

一小部分同分布的数据 + 小一点的模型 → 试一下 → 有用再扩大规模

例子

MAI 选 STEM 和编程占 56%，也是试出来的



课堂问答

补充 强化学习会自动按对错给样本加权

课上口头补充

奖励越多，梯度权重越高（粗细示意）



奖励高

奖励低

犯了错又恢复：错的那步也被强化

最常见的做法 ①

只筛正确的轨迹做监督微调

最常见的做法 ②

交给强化学习的算法重新加权



课堂问答

补充 按和下游任务对不对口来挑数据

课上口头补充

一种办法

拿评测集上的损失当信号

但只是近似

没在那批数据上完整训一遍

更依赖实验



课堂问答

Q6 训练目标是什么?

预训练

交叉熵

整段文本都算损失

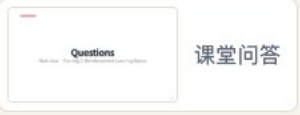
监督微调

同样是交叉熵

带角色标签的聊天格式

损失加一个掩码

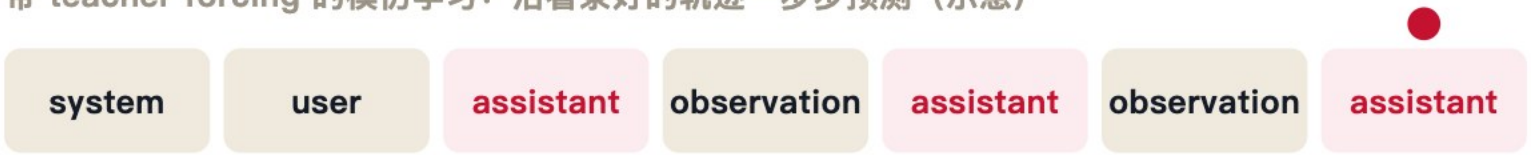
~~工具返回的结果~~ 通常不监督



课堂问答

Q7 有人在做模仿吗?

带 teacher forcing 的模仿学习：沿着录好的轨迹一步步预测（示意）

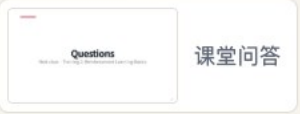


训练时

上下文固定，不从模型里采样

接下来几讲

模型自己产生训练数据



课堂问答

agent 的监督微调：五句话

② 改权重

新行为跟着每次调用走 · 代价：要训练 · 看不见 · 换不了模型 · 给强化学习做冷启动

③ token 序列

套上聊天模板 → 一串 token · 只让助手 token 算损失 · 结束符在掩码里 · 角色头不在 · 推理：留不留 · 训不训 · 配比按 token 看

④ 挑轨迹

通过测试的也会带坏习惯 · 至少 5 轮：平均有用 · 最强的模型 ≠ 最好的老师 · 任务来源影响最大 · 扩数据要加新任务

⑤⑥ 统一和训练

先统一再按脚手架渲染 · 混合多来源 > 单一领域 · 压住路由 · 打包不切对话 · 模板一字不差

⑦⑧ 验收和交接

先过拟合 20 个样本 · 在目标脚手架里评 · 领域外也要测 · 从故意没训到最好的检查点开始强化学习



课件 p.48